

Simulazione del comportamento di stormi

Massimo Prattichizzo
Francesco Manco
Francesca Mancinelli

7 febbraio 2026

Indice

1	Variazioni rispetto alla consegna precedente	1
2	Regole di volo	1
3	Strumenti	2
4	Implementazione	2
5	Compilazione e comandi	4
6	Testing	4

1 Variazioni rispetto alla consegna precedente

Per migliorare la leggibilità del codice, la struttura del programma è stata modificata, rendendola più modulare con l'utilizzo di funzioni.

Al programma è stata aggiunta la struct Vector in cui sono definiti vettori a due componenti e relativi metodi. Inoltre è stata implementata una classe Predator, che insieme a Fish eredita dalla classe base Boid. La nuova simulazione contiene quindi anche dei predatori comandati dall'utente. Per questo motivo alle regole di volo è stato aggiunto un fattore di fuga dei boids, oltre a uno che tiene conto del comportamento degli stessi ai bordi della finestra grafica. L'implementazione delle regole di volo è stata inoltre modificata per risolvere problemi che si presentavano nella versione precedente del progetto, ad esempio la sovrapposizione continua dei boids durante l'applicazione della regola di separazione.

L'interfaccia grafica è stata migliorata con l'utilizzo di una libreria GUI per rendere la simulazione più interattiva e visivamente interessante.

2 Regole di volo

Il programma simula il comportamento di oggetti paragonabili ad uno stormo di uccelli in volo, in questo caso chiamati *boids*. I boids seguono delle regole i cui parametri possono essere variati dall'utente al runtime.

Le regole si applicano tra due o più boids che non superano la distanza d tra loro. Dato un boid b_i e gli altri b_j , le regole sono:

- Separazione: il boid evita i vicini che si trovano ad una distanza $d_s < d$. Questa regola segue l'equazione

$$\vec{v}_{sep} = -s \sum_{i \neq j} (\vec{x}_{b_j} - \vec{x}_{b_i}) \quad (1)$$

- Allineamento: il boid si allinea alle traiettorie di volo dei vicini con una rapidità dipendente dal parametro a

$$\vec{v}_{ali} = -a \left(\frac{1}{n-1} \sum_{i \neq j} \vec{v}_{b_j} - \vec{v}_{b_i} \right) \quad (2)$$

- Coesione: il boid si muove verso il centro di massa dei vicini

$$\vec{x}_c = \frac{1}{n-1} \sum_{i \neq j} \vec{x}_{b_j} \quad (3)$$

da cui, dato il fattore c

$$\vec{v}_{coes} = c(\vec{x}_c - \vec{x}_{b_i}) \quad (4)$$

Oltre a queste, tipiche dell'algoritmo di Craig Reynolds, si aggiungono:

- Fuga: i boids che si trovano ad una distanza d_{pred} da uno o più predatori aumentano la propria velocità di un fattore $\vec{v}_{esc}$ linearmente con la distanza dal predatore.
- Comportamento ai bordi: quando i boids si avvicinano ai bordi della finestra, la loro velocità viene variata in modo da farli allontanare dagli estremi. Alla velocità si aggiunge quindi un valore $\vec{v}_{turn}$.

Il programma aggiorna quindi le posizioni e le velocità di ogni boid tenendo conto di queste regole, per cui:

$$\vec{x}_{b_i} = \vec{x}_{b_i} + \vec{v}_{b_i} \Delta t \quad (5)$$

$$\vec{v}_{b_i} = \vec{v}_{b_i} + \vec{v}_{sep} + \vec{v}_{ali} + \vec{v}_{cohes} + \vec{v}_{esc} + \vec{v}_{turn} \quad (6)$$

In questo caso si è considerato uno spazio bidimensionale, per cui i vettori posizione e velocità sono formati da due componenti.

3 Strumenti

Il programma è realizzato in *C++*, mentre per la componente grafica sono state utilizzate due librerie:

- *SFML*, in particolare la versione 2.6.2;
- la libreria cross-platform TGUI, versione 0.9.5, scaricabile con il seguente codice:

```
$ git clone https://github.com/texus/TGUI.git
$ cd TGUI
$ mkdir build
$ cd build
```

Se SFML 2.6 si trova in `/usr/local`:

```
$ cmake .. -DTGUI_BUILD_SHARED_LIBS=ON -DSFML_DIR=/usr/local/lib/cmake/SFML
$ cmake --build .
$ sudo cmake --install .
```

In tutti i file è stato applicato Clang Format, uno strumento automatico per la formattazione del codice. In particolare, viene usata la configurazione *google*.

La gestione dei file, del testing e il *build* del progetto sono invece affidati a *CMake*, i cui comandi sono contenuti nel file *CMakeLists.txt*. Questi servono a creare l'eseguibile del programma contenente i file source, aggiungere le librerie esterne *SFML* e *TGUI*, mostrare i warning durante la compilazione, attivare l'*address sanitizer* in modalità *Debug* e creare l'eseguibile per i test.

Per facilitare la collaborazione su un unico progetto e documentare le modifiche è stato utilizzato Git, attraverso la piattaforma online GitHub.

Sono stati utilizzati sistemi di Intelligenza Artificiale per la creazione dei file "*aquarium.jpeg*" e "*algae.png*", usati per lo sfondo della simulazione.

4 Implementazione

In *Vector.hpp* e *Vector.cpp* viene definita la struct *Vector*, di vettori a due componenti. Una classe *user-defined* per i vettori ha permesso di implementare operazioni utili all'aggiornamento di posizioni e velocità dei boids. In particolare, viene effettuato l'*overloading* di diversi operatori, permettendo operazioni tra vettori con se stessi e con scalari e vengono definite funzioni di normalizzazione, limite minimo e massimo sul modulo dei vettori e scalatura.

Nella classe *Boid*, contenuta in *Boid.hpp* e *Boid.cpp* si definisce invece il costruttore di un'entità di tipo boid, caratterizzata da due vettori bidimensionali: una contenente la

posizione e una la velocità del singolo oggetto. Questa viene usata come classe di base per i due tipi di boids, per cui viene richiesto che sia astratta definendone il distruttore come *virtual*.

In *Flock.hpp* e *Flock.cpp*, la classe *Flock* contiene come membri privati entità grafiche di *SFML* come la finestra e la texture dei boids. Oltre a questi, sono definiti due *std :: vector* contenenti i puntatori degli oggetti di tipo *Fish* e *Predator*. Il numero di oggetti da aggiungere ai due vettori viene impostato dall'utente durante la compilazione e passato al metodo *Initialize()* che chiama ricorsivamente *addFish()* e *addPredators()*, i quali generano numeri casuali da assegnare inizialmente alle componenti di posizione dei boids e velocità dei fish, mentre la velocità iniziale dei predatori è nulla.

Per la generazione di numeri pseudocasuali è stata usata la classe *std :: default_random_engine* con una distribuzione uniforme. Il range delle posizioni è compreso tra 0 e il valore massimo della componente considerata della finestra, ottenuta con il metodo *sf :: Window :: getSize()*. Le velocità, invece sono comprese nell'intervallo $2\pi \cdot [-100, 100]$. Questi valori di velocità permettono spostamenti sulla finestra che non siano troppo grandi, in modo da poter applicare localmente le regole, ma comunque osservabili. Considerando infatti la gestione di *delta_t*, discussa in seguito, il suo valore risulta dell'ordine di $10^{-2}s$, per cui lo spostamento iniziale massimo di un boid è dell'ordine di $100 \cdot 10^{-2} = 1$ pixel ad ogni step.

Infine, il metodo *Update()* aggiorna le posizioni e le velocità dei boids attraverso i metodi *run()* e *render()*, discussi successivamente. I parametri iniziali delle regole di volo sono definiti in *Flock*, ma la maggior parte di questi possono essere variati al runtime.

Una delle classi derivate di *Boid* è *Fish*, nella quale sono implementate le regole di volo dello stormo. Per ognuna, un ciclo *for* controlla che sia soddisfatta la condizione sulla distanza dagli altri boids e, in caso positivo, applica la formula corrispondente. Per regolare il comportamento dei boids, viene invocato il metodo di *Vector* per limitare al valore *maxForce* il modulo di *steering*, cioè della velocità della regola corrispondente.

La fuga dai predators è rappresentata dalla funzione *escape()*, che ha una struttura analoga alle altre regole. In questo caso la velocità v_{esc} , essendo lineare con la distanza tra fish e predatore, viene aggiornata esattamente con questo valore. In questo caso si è scelto un valore limite maggiore di quello delle altre regole in modo da rendere evidente la fuga dai predatori.

La funzione *borders()* gestisce il comportamento dei boids ai bordi della finestra, per cui quando questi iniziano ad avvicinarsi ai margini, in particolare a una distanza di 200 pixel da essi, la componente di v_{turn} relativa al bordo considerato viene aggiornata di un *turnfactor* in verso opposto alla direzione del boid. Nel caso limite in cui il boid si trovi esattamente sul bordo, la sua posizione viene aggiornata in modo da riposizionarlo su di esso e non farlo uscire dalla finestra della simulazione.

La seconda classe derivata da *Boid* è *Predator*. I predatori hanno un metodo *separate()*, nella struttura del tutto analogo a *separation()* dei fish. Questo permette ai predatori, i quali seguono tutti il cursore, di non sovrapporsi né durante il movimento né quando questo risulta fermo.

Sia *Fish* che *Predator* contengono due metodi fondamentali:

- *run()*, che per i fish aggiorna la velocità aggiungendo i contributi delle regole di volo e calcola quindi la posizione. Per i predatori, ne aggiorna la velocità, e quindi la posizione, calcolandola con l'ausilio del metodo *sf :: Mouse :: getPosition()*. Lo

sprite del predatore non coincide esattamente con il cursore in modo garantire un movimento fluido.

- *render()*, che crea uno sprite a partire dalla texture fornita, ne imposta l'origine nel centro della texture e gestisce l'orientamento ribaltando lo sprite in base alla direzione del boid. Aggiorna inoltre la posizione dello sprite con quella del boid corrispondente e attraverso *sf :: RenderWindow :: draw()* lo disegna sulla finestra.

In *main.cpp* si gestisce l'inizializzazione del numero di boids, l'interfaccia grafica e la finestra. In particolare, con il tema *Black.txt* della libreria TGUI viene creato un panel contenente gli slider dei parametri di volo. Questi vengono impostati attraverso la funzione *createWidget* e aggiornano le variabili della simulazione al runtime grazie alla funzione *onValueChanged*. Si caricano inoltre le immagini relative allo sfondo e all'ambientazione su vari livelli. In un ciclo *while*, corrispondente all'apertura della finestra, viene utilizzata la classe *sf :: Clock* per misurare il tempo fra due frame consecutivi. Questo valore viene utilizzato come *delta_t* in modo da avere un tempo reale e uno spostamento dei boids coerente indipendentemente dal frame rate del dispositivo. Come discusso precedentemente, considerando che un frame rate standard si aggira intorno ai 60 FPS, il *delta_t* atteso è dell'ordine di 10^{-2} . All'interno di questo ciclo viene inoltre chiamata la funzione *Update()*, che aggiorna la dinamica del sistema. Infine, il frame viene visualizzato con *display()*.

5 Compilazione e comandi

Per compilare il programma sono richiesti *CMake*, *git* e *g++* oltre che le librerie esterne *SFML* e *TGUI*.

I comandi da lanciare per la compilazione in *Debug mode* sono elencati di seguito:

```
$ cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug $ cmake --build build
```

Per far partire il programma si usa quindi:

```
$ build/progetto
```

Per eseguire la simulazione il compilatore richiede in input il numero di fish e numero di predatori. I valori consentiti vanno da un minimo di 0, e un massimo, rispettivamente, di 200 e 5.

L'utente può interagire con la simulazione in due modi: comandando il movimento dei predatori con il cursore, e modificando alcuni dei parametri al runtime attraverso gli slider presenti nel pannello in alto a sinistra sulla finestra grafica. Gli slider tengono automaticamente conto del fatto che i valori che può assumere la *Separation distance* (d_s) devono essere minori del *Visual range* (d), per cui il massimo valore che assume lo slider di d_s è impostato a un valore positivo minore di un fattore 100 rispetto a d .

La simulazione può essere messa in pausa in ogni momento premendo la Spacebar sulla propria tastiera.

6 Testing

Il programma è stato testato nel file *Test.cpp* attraverso l'utilizzo di *doctest.h*.

Il primo test si occupa di verificare che *addFish()* imposti correttamente la posizione e la velocità dei fish.

Nel secondo e terzo test viene verificato che i vettori di fish e predatori contengano il numero corretto di entità richieste.

Nel terzo test si verificano le tre regole di volo classiche con diversi *subcases*. Viene quindi considerato come parametro non nullo solo quello associato alla regola richiesta e, con due fish, si verifica che posizione e velocità del primo dei due venga modificata come atteso.

Per il running dei test si usa il comando:

```
$ build/progetto.t
```